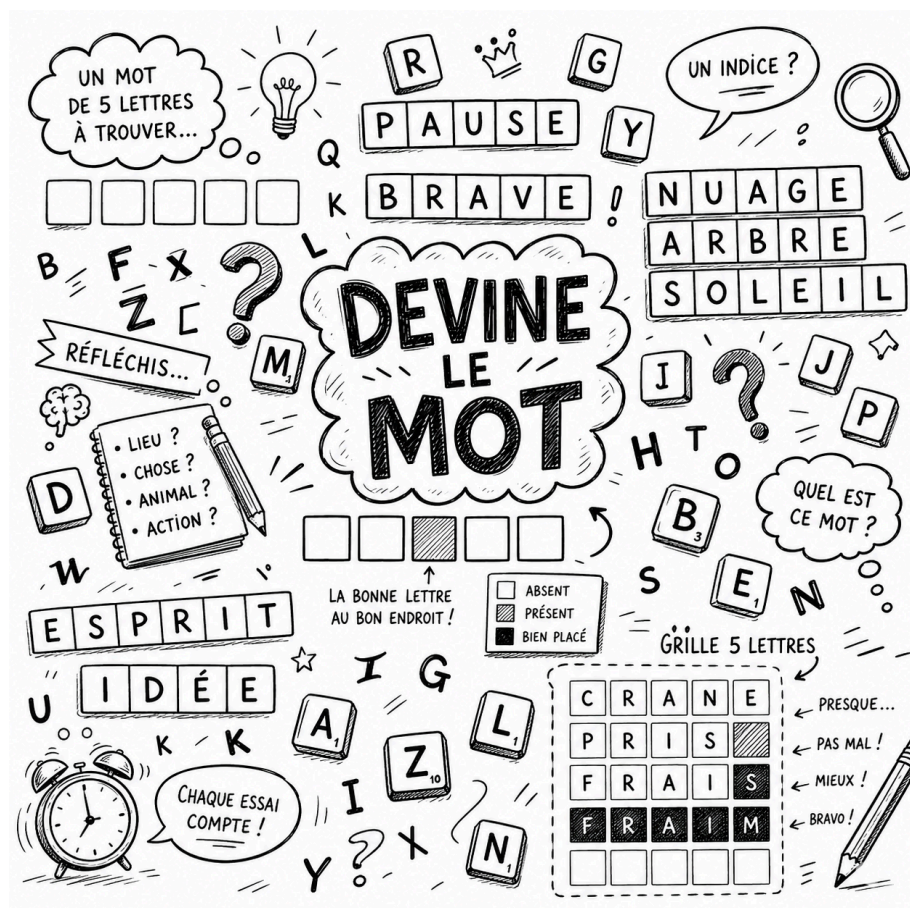


# SAÉ R201 – Base de la programmation objet

## Find My Word

Groupe : Abou Elela Zyad, Hamou Yahia Achour

|                                        |    |
|----------------------------------------|----|
| 1. Organisation du travail.....        | 2  |
| 2. Diagramme de classes simplifié..... | 3  |
| 3. Diagramme de classes détaillés..... | 4  |
| 4. Choix d'architecture.....           | 5  |
| 5. Comment lancer le programme.....    | 6  |
| 6. Bilan technique.....                | 7  |
| 7. Cas test.....                       | 8  |
| 8. Conclusion.....                     | 10 |



## Introduction:

Dans le cadre de notre formation en première année de BUT informatique, nous avons pu étudier le développement orienté objet à travers divers langages dont Java. L'occasion est venue de mettre en pratique les connaissances que nous avons acquises dans un projet concret.

Ce projet consiste à développer en Java un jeu de devinette de mots appelé FindMyWord, inspiré du jeu Wordle. L'objectif est de permettre à un joueur de deviner un mot secret de 5 lettres en 6 tentatives maximum. Après chaque tentative, le programme indique pour chaque lettre si elle est bien placée (OK), mal placée (PRÉSENTE) ou absente du mot secret (ABSENTE). Le projet a été réalisé en binôme dans le cadre de la SAE 201.

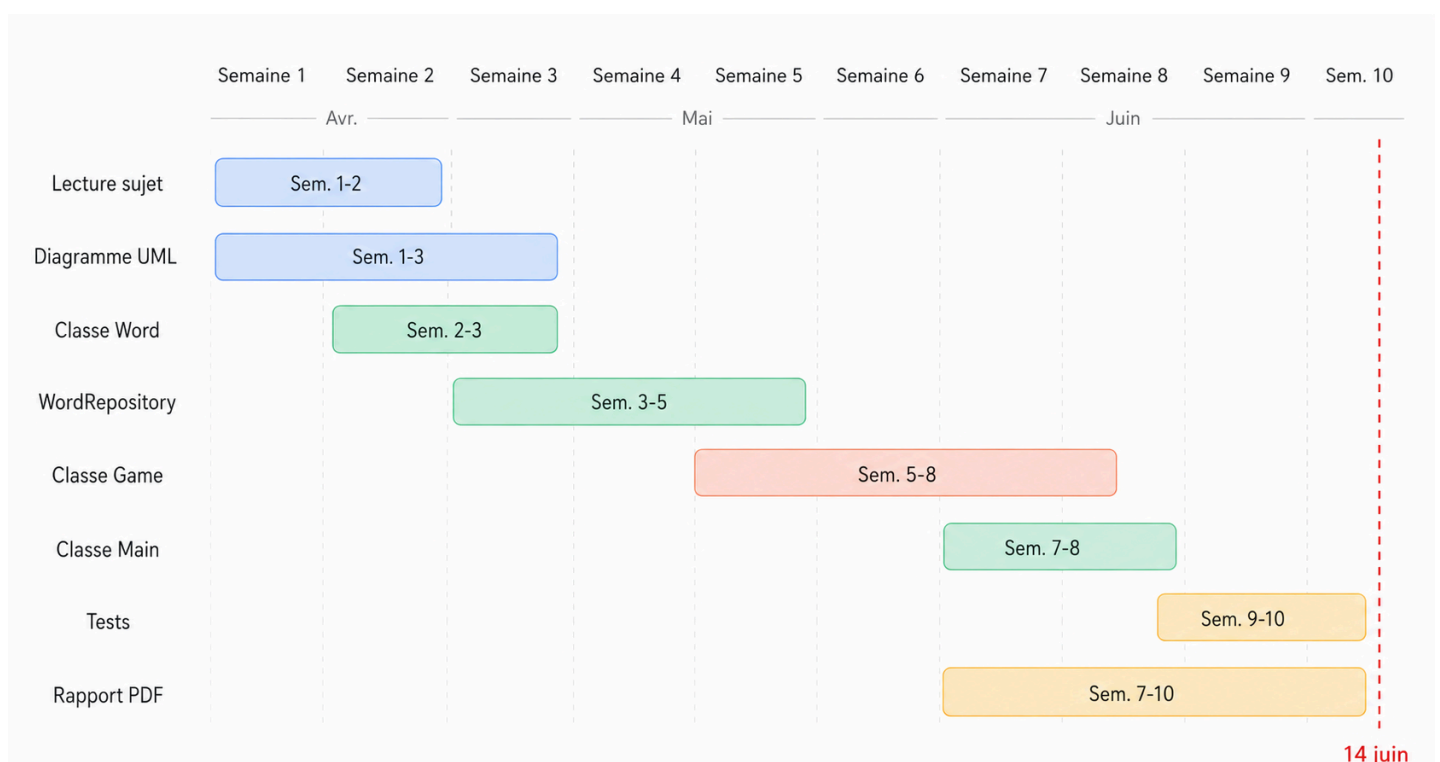
## 1. Organisation du travail

### Organisation du travail prévue

Avant de commencer le développement, nous avons défini ensemble une bonne répartition des tâches. Achour devait se charger de la modélisation et de la gestion des mots, tandis que Zyad devait s'occuper de la logique du jeu et de l'interface utilisateur. Les classes les plus complexes, notamment la classe Game, elles devaient être réfléchies et conçues ensemble avant toute implémentation, afin d'assurer la cohérence de l'ensemble du projet.

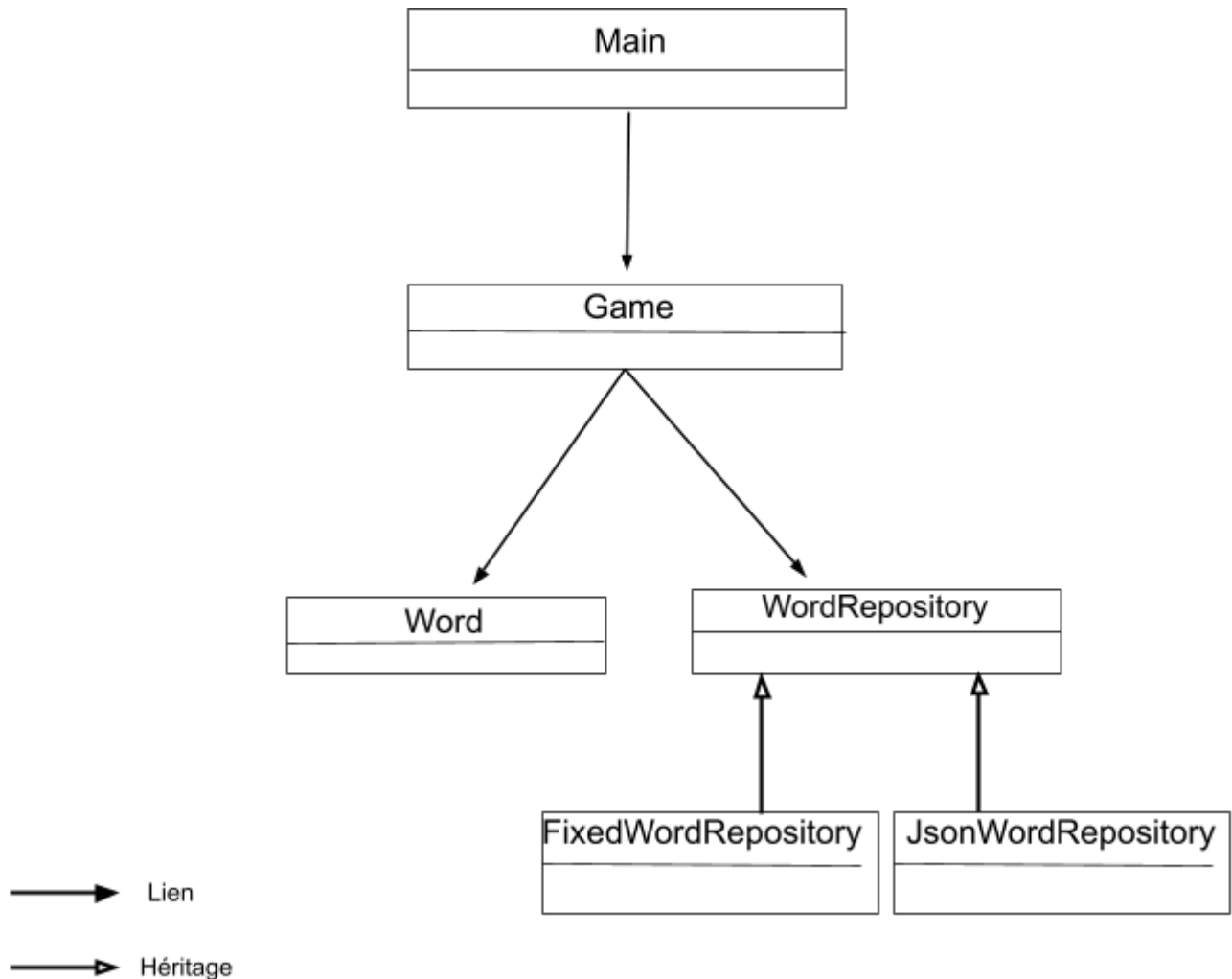
Outils utilisés : Discord, Google Docs, Google Drive, VsCode, Lucidchart

### Diagramme de GANTT:



## 2. Diagramme de classes simplifié

Premier diagramme de classes simplifié des classes

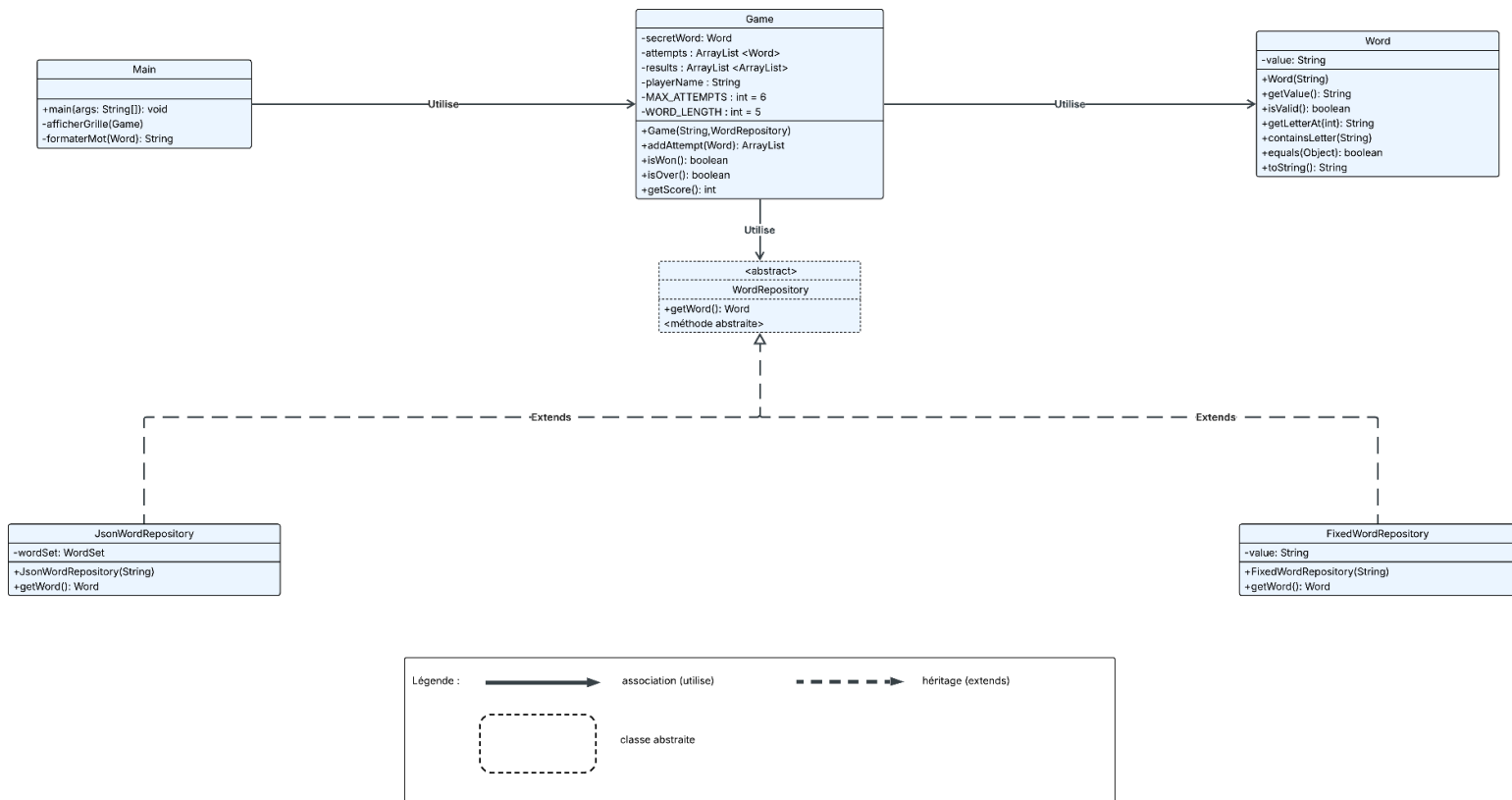


(où *JsonWordRepository* est une classe qui récupère un mot aléatoire )

### Description :

Le diagramme présente six classes reliées entre elles. La classe **Main** est le point d'entrée du programme, elle est associée à la classe **Game** qui gère toute la logique du jeu. **Game** est elle-même associée à deux classes : **Word** qui représente un mot du jeu, et **WordRepository** qui est la source de mots. **WordRepository** est une classe abstraite dont héritent deux classes concrètes : **FixedWordRepository** qui retourne toujours le même mot (utile pour les tests), et **JsonWordRepository** qui pioche un mot aléatoire dans le fichier **mots.json**.

### 3. Diagramme de classes détaillés



(voici le lien vers l'image : [Diagramme\\_UML\\_FindMyWord.png](#) )

## 4. Choix d'architecture

Pour représenter les différentes sources de mots, nous avons créé une classe abstraite `WordRepository` plutôt qu'une interface. Ce choix vient du fait qu'une classe abstraite nous permet de partager du code entre les classes filles si besoin. `JsonWordRepository` et `FixedWordRepository` héritent toutes les deux de cette classe abstraite et fournissent chacune leur propre façon de retourner un mot. Grâce à ce système, la classe `Game` n'a pas besoin de savoir d'où vient le mot secret : elle demande simplement un mot à son `WordRepository` et c'est tout. Pour passer du mode test au mode jeu réel, une seule ligne change dans `Main`.

Pour la classe `Word`, nous avons choisi de regrouper tout les tests à l'intérieur de la classe plutôt que de les éparpiller ailleurs dans le code. Ainsi `Game` n'a pas besoin de savoir comment un mot est vérifié, elle pose juste la question et `Word` répond. Nous avons aussi ajouté des méthodes comme `getLetterAt()` et `containsLetter()` pour que `Game` puisse analyser les tentatives sans jamais toucher directement aux données internes de `Word`.

Pour la classe `Game`, nous avons stocké à la fois les tentatives et leurs résultats dans deux listes séparées. Cela permet à `Main` d'afficher la grille complète à n'importe quel moment sans avoir à recalculer les résultats. L'algorithme d'analyse `OK / PRESENT / ABSENT` est une méthode privée de `Game` car c'est une opération interne qui ne doit pas être accessible depuis l'extérieur.

Nous aurions pu utiliser des tableaux classiques dans la classe `Game`, pour stocker les tentatives et les résultats, puisque le nombre maximum d'essais est fixé à 6 (Comme conseiller en cours). Cependant nous avons préféré utiliser des `ArrayList` car elles gèrent automatiquement la taille réelle des données sans avoir à faire des compteurs séparés. À chaque ajout, `ArrayList` sait exactement combien d'éléments elle contient, ce qui simplifie le code et évite les erreurs liées aux cases vides.

Enfin, tout l'affichage a été mis dans `Main` avec des méthodes dédiées comme `afficherGrille()` et `formaterMot()`. Nous avons également décidé de continuer la partie si 2 conditions sont réunies : 1/ le joueur a gagné la manche 2/ le joueur souhaite continuer ainsi qu'un mode test qui permet de tester le jeu en choisissant son propre mot.

## 5. Comment lancer le programme

### 1. Aller dans le dossier Abou-Elela\_Hamou-Yahia depuis le terminal :

```
cd chemin_vers_notre_dossier/Abou-Elela_Hamou-Yahia
```

### 2. Compiler :

```
javac -cp wordset.jar findmyword\*.java
```

### 3. Lancer :

```
java -cp "wordset.jar;." findmyword.Main
```

## Structure

```
Abou-Elela_Hamou-Yahia/
├── findmyword/
│   ├── Word.java
│   ├── WordRepository.java
│   ├── JsonWordRepository.java
│   ├── FixedWordRepository.java
│   ├── Game.java
│   └── Main.java
├── test
│   └── ...
├── wordset.jar
├── mots.json
└── README.md
```

Le projet utilise une bibliothèque externe (wordset.jar) fournie par le sujet pour charger les mots depuis mots.json. L'option -cp indique à Java où chercher les classes dont il a besoin : lors de la compilation on précise wordset.jar pour trouver les classes de la bibliothèque, et \*.java pour compiler tous nos fichiers en une seule commande. Lors de l'exécution, le classpath contient wordset.jar pour la bibliothèque et . pour nos propres classes compilées. Sans l'un ou l'autre, Java ne trouverait pas ce dont il a besoin et le programme ne démarrerait pas. Ces commandes doivent être exécutées depuis la racine du projet où se trouvent tous les fichiers nécessaires.

## Lancer les fichier Tests:

1. Aller dans le dossier Abou-Elela\_Hamou-Yahia depuis le terminal :

```
cd chemin_vers_notre_dossier/Abou-Elela_Hamou-Yahia
```

2. Compiler :

```
javac -cp "wordset.jar;." findmyword\*.java test\*.java (compile toute les classe)
```

3. Lancer :

```
java -cp "wordset.jar;." test.TestWord  
java -cp "wordset.jar;." test.TestGame  
java -cp "wordset.jar;." test.TestWordRepository  
java -cp "wordset.jar;." test.TestFixedWordRepository  
java -cp "wordset.jar;." test.TestJsonWordRepository
```

## 6. Bilan technique

### Ce qui a été fait :

- la classe Word avec validation complète (longueur, caractères alphabétiques, lettres répétées)
- La classe abstraite WordRepository avec deux classes filles :  
JsonWordRepository (mot aléatoire depuis mots.json) et  
FixedWordRepository (mot fixe pour les tests)
- La classe Game avec l'analyse OK / PRESENT / ABSENT, la détection victoire/défaite et le calcul du score
- Enfin, la classe Main avec la boucle de jeu, l'affichage de la grille, la gestion des erreurs de saisie et l'accumulation des scores sur plusieurs manches  
La possibilité de rentrer en mode test et de choisir son mot
- Un fichier test pour chaque classes

### Ce qui fonctionne:

→ Compilation et exécution depuis le terminal

- Chargement du fichier mots.json via wordset.jar
- Validation des tentatives avec messages des messages d'erreur distincts
- Analyse OK / PRESENT / ABSENT
- Affichage de la grille mise à jour après chaque tentative
- Détection de la victoire et de la défaite
- Calcul du score par manche
- Accumulation des scores sur plusieurs manches
- Le joueur peut choisir de continuer ou d'arrêter après chaque victoire
- Test par classes fonctionnent

**Ce qui n'a pas été fait :**

- Mode deux joueurs (optionnelle)
- Gestion du temps avec un timer (optionnelle)
- Historique des parties (optionnelle)

## 7. Cas test

```
PS C:\Users\zyad > cd C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> javac -cp wordset.jar findmyword\*.java
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> java -cp "wordset.jar;" findmyword.Main
=====
FIND MY WORD - BUT1
=====
Entrez votre nom : Zyad
```

```
PS C:\Users\zyad_> cd C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> javac -cp "
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> java -cp "
=====
FIND MY WORD - BUT1
=====
Entrez votre nom : Zyad
Bonjour Zyad !
Mode test ? (o/n) : 
```

```
PS C:\Users\zyad_> cd C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> javac -cp wordset.jar findmyword\*.java
PS C:\Users\zyad\Desktop\Abou-Elela_Hamou-Yahia_FindMyWord> java -cp "wordset.jar;" findmyword.Main
=====
                FIND MY WORD - BUT1
=====
Entrez votre nom : Zyad
Bonjour Zyad !
Mode test ? (o/n) : o
Entrez le mot secret : chine
```

[illegible]

Tentative 1 : poire

-----

```
[ P ][ O ][ I ][ R ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
```

Tentative 2 :

```
-----
[ P ][ O ][ I ][ R ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ B ][ R ][ I ][ S ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ C ][ R ][ I ][ S ][ E ] ? OK ABSENT OK ABSENT OK
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
```

Tentative 4 :

Tentative 4 : huile

-----

```
[ P ][ O ][ I ][ R ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ B ][ R ][ I ][ S ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ C ][ R ][ I ][ S ][ E ] ? OK ABSENT OK ABSENT OK
[ H ][ U ][ I ][ L ][ E ] ? PRESENT ABSENT OK ABSENT OK
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
```

Tentative 5 :

Tentative 5 : chine

-----

```
[ P ][ O ][ I ][ R ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ B ][ R ][ I ][ S ][ E ] ? ABSENT ABSENT OK ABSENT OK
[ C ][ R ][ I ][ S ][ E ] ? OK ABSENT OK ABSENT OK
[ H ][ U ][ I ][ L ][ E ] ? PRESENT ABSENT OK ABSENT OK
[ C ][ H ][ I ][ N ][ E ] ? OK OK OK OK OK
[ ][ ][ ][ ][ ] ? ***** ***** ***** ***** *****
```

Bravo Ziad !

Vous avez trouvé le mot en 5 essais.

Score cette manche : 2 points

Score total : 2 points

Voulez-vous continuer ? (o/n) :

Voulez-vous continuer ? (o/n) :

n

=====

Score final : 2 points

Merci d'avoir joué Ziad !

PS C:\Users\zyad\Desktop\Abou-Elela Hamou-Yahia FindMyWord>

## 8. Conclusion

En pratique, le déroulement a été différent de ce que nous avions prévu.

Les deux premières séances se sont déroulées comme prévu. La classe `Word` a été réalisée sans difficulté majeure, de même que `WordRepository` et ses deux classes filles. La prise en main de la bibliothèque externe `wordset.jar` a cependant pris plus de temps que prévu : il a fallu comprendre comment l'intégrer au classpath lors de la compilation et de l'exécution depuis le terminal, ce que nous n'avions jamais fait avant.

La classe `Game` a été la plus difficile à implémenter. L'algorithme d'analyse OK / PRESENT / ABSENT, la gestion des `ArrayList` pour stocker les tentatives et les résultats, ainsi que la détection des situations de fin de partie ont nécessité beaucoup plus de temps que ce que nous avions prévu. Nous avons tous les deux dû travailler en même temps sur cette classe, ce qui a décalé la répartition initiale du travail.

La classe `Main` a également demandé un effort de plus pour bien séparer l'affichage de la logique métier et pour gérer proprement les différents messages d'erreur selon le type de saisie invalide.

Nous tenons à vous remercier, M. Bouthinon, pour votre accompagnement et vos conseils tout au long de l'année, et nous vous souhaitons une belle continuation !

Abou Elela Zyad, Hamou Yahia Achour